

COMPTE-RENDU TECHNIQUE

Analyse de conversations patient-thérapeute sur la santé mentale

Auteur : LARROUX Logan

Encadrante : Lynda Tamine-Lechani

Équipe :

*Ulysse Chasseigne, Idir Yahiaoui, Bantsoukissa Laure,
Cruz Fernandes Diogo, Emin Belkheir, Esso-Y-N Trésor PANA,
Logan Larroux, Yvan Sellier, Victor Blanquart-Blanchet, Lucas Da Silva*

18 mai 2026



[Info5.BE-SHS]

Données : [Kaggle — NLP Mental Health Conversations](#)

[Kaggle — Sentiment Analysis for Mental Health](#)

Code : [GitHub — branche main](#)

Table des matières

1	Contexte et objectifs	2
2	Qu'est-ce que BERT ?	3
2.1	Définition générale	3
2.2	Étapes d'apprentissage	3
2.2.1	1. Pré-entraînement (non supervisé)	3
2.2.2	2. Fine-tuning supervisé	4
2.2.3	Tokenisation spécifique : WordPiece	4
2.3	Points forts de BERT	5
2.4	Limites de BERT	5
2.4.1	Coût computationnel élevé	5
2.4.2	Limite de longueur des séquences	5
3	Initialisation commune aux trois modèles	6
3.1	Dépendances	6
3.2	Préparation des données	6
3.3	Découpage des données	6
3.4	Encodage des labels	7
4	Modèle A — BERT sans fine-tuning (extraction de features)	8
4.1	Principe et architecture	8
4.2	Tokenisation BERT	8
4.3	Extraction des embeddings [CLS]	8
4.4	Classification avec un classifieur léger	9
4.5	Évaluation	9
5	Modèle B — BERT avec fine-tuning	9
5.1	Principe du fine-tuning	9
5.2	Préparation du Dataset PyTorch	9
5.3	Chargement du modèle et tête de classification	10
5.4	Boucle d'entraînement	10
5.5	Évaluation	11
6	Modèle C — BERT spécialisé + fine-tuning	11
6.1	Pourquoi un modèle spécialisé ?	11
6.2	Modèles disponibles sur Hugging Face	11
6.3	Adaptation de la tête de classification	12
6.4	Fine-tuning et évaluation	12
7	Comparaison globale des modèles	12
7.1	Tableau récapitulatif	12
7.2	Questions d'analyse	13
7.3	Points de vigilance	13
	Références	14

Contexte et objectifs

Ce document est un complément à l'étape 2 du projet. Tandis que la partie principale de l'étape 2 portait sur des méthodes classiques de classification (Naive Bayes, Régression Logistique, LinearSVC en supervisé ; K-Means en non supervisé), ce notebook extra introduit une approche radicalement différente reposant sur **BERT** (*Bidirectional Encoder Representations from Transformers*).

L'objectif reste identique : produire un classificateur de sentiments sur des textes liés à la santé mentale, en s'appuyant sur le même corpus d'entraînement (`Combined_Data.csv` — *Sentiment Analysis for Mental Health*, Kaggle¹).

Ce qui change, c'est **la profondeur de la représentation** : là où TF-IDF ne comptait que des mots, BERT modélise le langage dans toute sa complexité contextuelle.

Trois variantes de BERT sont implémentées et comparées entre elles, ainsi qu'avec les résultats du notebook principal :

Modèle	Nom court	Description
A	BERT sans fine-tuning	Extraction de features, poids gelés
B	BERT avec fine-tuning	Tous les poids mis à jour sur notre dataset
C	BERT spécialisé + fine-tuning	Modèle pré-entraîné sur des sentiments (<i>Hugging Face</i>) puis fine-tuné

Remarque

Les métriques d'évaluation utilisées sont identiques à celles de l'étape 2 (accuracy, classification report, matrice de confusion) afin de permettre une comparaison directe.

1. lien du dataset : <https://www.kaggle.com/datasets/suchintikasarkar/sentiment-analysis-for-mental-health>

Qu'est-ce que BERT ?

Définition générale

BERT (*Bidirectional Encoder Representations from Transformers*) est un modèle de langage développé par Google en 2018. Il repose sur l'architecture **Transformer**, mais n'en utilise que la partie **encodeur**.

Sa particularité fondamentale est d'être **bidirectionnel** : contrairement aux modèles précédents qui lisaient le texte uniquement de gauche à droite (ou de droite à gauche), BERT lit le contexte dans les deux directions simultanément.

Exemple d'ambiguïté résolue par la bidirectionnalité

« Le chat mange la souris »

BERT comprend le mot *mange* en regardant à la fois *chat* (contexte gauche) et *souris* (contexte droit) en une seule passe — ce qu'un modèle unidirectionnel ne peut pas faire.

Étapes d'apprentissage

L'apprentissage de BERT se déroule en deux phases distinctes.

1. Pré-entraînement (non supervisé)

Avant toute tâche spécifique, BERT est pré-entraîné sur d'énormes corpus (Wikipedia anglophone, BooksCorpus, etc.) via deux tâches auto-supervisées :

- **Masked Language Model (MLM)** : environ 15 % des tokens sont masqués aléatoirement et le modèle doit les prédire à partir du contexte. Exemple : « Le [MASK] mange la souris » → prédit *chat*.
- **Next Sentence Prediction (NSP)** : le modèle apprend à déterminer si deux phrases se suivent logiquement dans un texte ou sont juxtaposées aléatoirement.

Cette phase est entièrement non supervisée : aucune annotation humaine n'est requise. Elle permet à BERT d'acquérir une compréhension riche et générale du langage.

2. Fine-tuning supervisé

C'est ici qu'intervient l'apprentissage **supervisé**. On reprend le BERT pré-entraîné et on l'adapte à une tâche précise avec des données étiquetées, en ajoutant une petite couche de sortie :

Tâche	Exemple	Couche ajoutée
Classification de texte	Sentiment positif/négatif	Dense sur le token [CLS]
NER	Identifier noms, lieux...	Dense sur chaque token
Question-Réponse	Extraire une réponse d'un passage	Deux classifieurs (début/fin)
Similarité de phrases	Phrases équivalentes ?	Dense sur [CLS]

Le fine-tuning ne nécessite que peu de données étiquetées et peu d'époques, car BERT a déjà appris une représentation riche du langage lors du pré-entraînement.

Tokenisation spécifique : WordPiece

BERT utilise un **tokeniseur WordPiece** : les mots rares ou inconnus sont découpés en sous-mots, ce qui garantit que tout vocabulaire peut être représenté.

```
"unhappiness" -> ["un", "##happi", "##ness"]
```

Deux tokens spéciaux sont systématiquement ajoutés :

- [CLS] en début de séquence : sa représentation finale encode l'information globale de la phrase et sert d'entrée à la tête de classification.
- [SEP] pour séparer deux segments distincts.

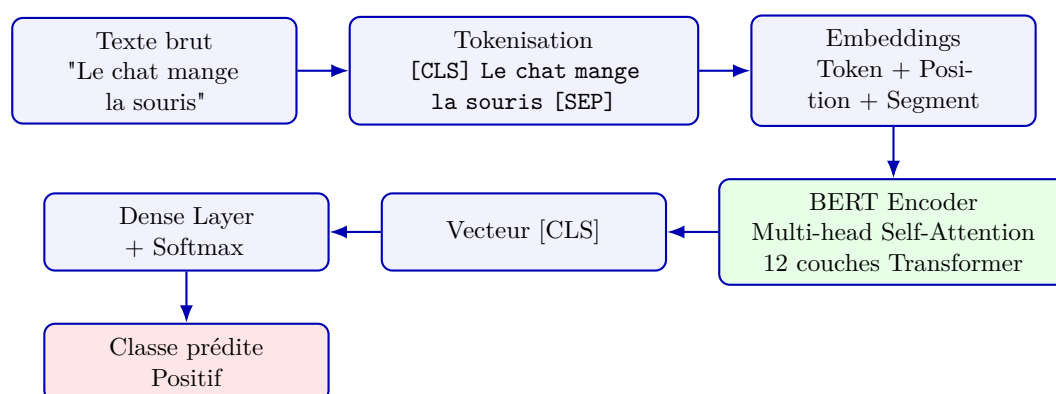


FIGURE 1 – Schéma simplifié du pipeline de BERT pour la classification de texte.

Points forts de BERT

- **Représentations contextuelles** : le même mot reçoit des vecteurs différents selon son contexte — *banque* n'est pas la même chose dans « banque financière » et « banque de sable ».
- **Transfert de connaissances efficace** : un BERT pré-entraîné une seule fois peut être fine-tuné pour des dizaines de tâches différentes avec peu de données.
- **Écosystème riche** : des variantes existent pour de nombreux besoins :
 - **RoBERTa** (plus robuste),
 - **CamemBERT** (français),
 - **DistilBERT** (plus léger et rapide),
 - etc.

Limites de BERT

Coût computationnel élevé

C'est probablement la limitation la plus concrète dans notre contexte :

- BERT-base : 110 millions de paramètres.¹
- BERT-large : 340 millions de paramètres.
- Le mécanisme d'attention est en $\mathcal{O}(n^2)$ par rapport à la longueur de la séquence : le coût explose sur les textes longs.
- Le fine-tuning et l'inférence nécessitent un GPU pour des temps raisonnables.

Limite de longueur des séquences

BERT est limité à **512 tokens** en entrée. Au-delà, le texte doit être tronqué ou découpé, ce qui peut faire perdre du contexte.

Variantes pour les textes longs

Des modèles comme **Longformer** ou **BigBird** ont été conçus pour pallier cette limitation via des mécanismes d'attention clairsemée (*sparse attention*).

1. Ces données ne sortent pas d'un chapeau, elle viennent du document officiel publié par Google : [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#) Section 3 : Model Architecture

Initialisation commune aux trois modèles

Dépendances

Les trois modèles partagent les mêmes bibliothèques :

Bibliothèque	Rôle
<code>transformers</code> (Hugging Face)	Chargement des modèles BERT et tokeniseurs pré-entraînés
<code>torch</code>	Framework de deep learning (entraînement et inférence)
<code>torch.utils.data</code>	<code>Dataset</code> et <code>DataLoader</code> pour les batchs
<code>sklearn</code>	Métriques, encodage des labels, découpage train/test
<code>numpy</code> , <code>pandas</code>	Manipulation des données
<code>matplotlib</code> , <code>seaborn</code>	Visualisation

Prérequis GPU

BERT est très coûteux en calcul. L'utilisation d'un GPU est fortement recommandée (Google Colab avec runtime GPU, ou machine locale équipée). La disponibilité se vérifie via `torch.cuda.is_available()`. Sur CPU, compter 2 à 3 heures par epoch^a d'entraînement.

^a. L'epoch est le nombre de passage complet sur l'ensemble des données

Préparation des données

Le pipeline de nettoyage est identique à celui du notebook principal : chargement de `Combined_Data.csv`, mélange, suppression des NaN, suppression des conflits d'étiquettes, application de la fonction `cleaner`.

Important : la tokenisation NLTK et la vectorisation BoW/TF-IDF/Word2Vec ne sont **pas** réappliquées. BERT dispose de son propre tokeniseur (`BertTokenizer`) qui opère directement sur le texte nettoyé.

Découpage des données

Contrairement au notebook principal (split 60/20/20), un découpage **80 %/20 %** train/test suffit ici. Le fine-tuning de BERT converge rapidement (2 à 3 epochs) et ses hyperparamètres sont peu nombreux, ce qui rend l'ensemble de validation optionnel.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
# Train size: 42 105 | Test size: 10 527
```

Encodage des labels

PyTorch exige des entiers comme labels. Un `LabelEncoder` de `sklearn` convertit les étiquettes textuelles en indices numériques. Il est fitté uniquement sur `y_train` (pas de *data leakage*) puis appliqué en `.transform` sur `y_test`.

```
encoder = LabelEncoder()
y_train_enc = encoder.fit_transform(y_train)
y_test_enc = encoder.transform(y_test)
# Classes: ['Anxiety', 'Bipolar', 'Depression', 'Normal',
#           'Personality disorder', 'Stress', 'Suicidal']
# num_labels = 7
```

Modèle A — BERT sans fine-tuning (extraction de features)

Principe et architecture

Dans cette première approche, **les poids de BERT sont gelés** — ils ne sont pas mis à jour pendant l'entraînement. BERT est utilisé comme un **extracteur de features** : il produit des représentations vectorielles denses pour chaque texte, et un classifieur léger (régression logistique) apprend à partir de ces vecteurs.

Texte brut $\xrightarrow{\text{BertTokenizer}}$ Tokens + masques $\xrightarrow{\text{BertModel (gelé)}}$ Embedding [CLS] (768 dim.) $\xrightarrow{\text{LogisticRegression}}$ Classe prédite

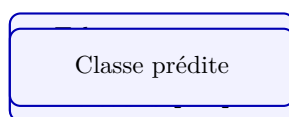


FIGURE 2 – Schéma simplifié du pipeline de BERT pour la classification de texte.

Avantages	Inconvénients
Très rapide (pas de rétropropagation dans BERT)	Performances souvent inférieures au fine-tuning
Peu de mémoire GPU nécessaire	BERT n'est pas adapté à la tâche spécifique
Sert de baseline solide et reproductible	Les embeddings [CLS] sans fine-tuning sont génériques

Tokenisation BERT

Le `BertTokenizer` convertit chaque texte en identifiants de tokens compréhensibles par BERT. Il gère automatiquement l'ajout des tokens spéciaux [CLS] et [SEP], le padding, la troncature à `max_length` tokens, et la création des `attention_mask` (1 pour les vrais tokens, 0 pour le padding).

Le paramètre `max_length=128` est un compromis entre couverture du texte et coût computationnel (la limite absolue de BERT est 512).

Extraction des embeddings [CLS]

On fait passer les textes tokenisés à travers `BertModel` en mode inférence (`torch.no_grad()`) pour récupérer le vecteur associé au token [CLS] (première position de la dernière couche cachée). Ce vecteur de 768 dimensions constitue la représentation globale du texte.

Rôle du token [CLS]

Le token [CLS] est placé en début de chaque séquence BERT. Dans les modèles fine-tunés, sa représentation encode l'information globale de la phrase et sert directement d'entrée à la tête de classification. Sans fine-tuning, il capture une représentation générale apprise sur Wikipedia/BooksCorpus — utile mais non spécialisée.

Classification avec un classifieur léger

Les embeddings extraits sont des vecteurs numériques de dimension 768 — exactement le format attendu par les classifieurs `sklearn`. Une régression logistique est utilisée ici pour sa rapidité et son efficacité sur des features de haute dimension.

D'autres classifieurs seraient envisageables : SVC, Random Forest, ou un petit réseau dense. La régression logistique reste le choix de référence pour sa simplicité et son interprétabilité.

Évaluation

Les métriques calculées sont identiques à celles de l'étape 2 : accuracy, classification report (precision, recall, F1-score par classe), matrice de confusion. Les résultats seront renseignés à l'issue de l'implémentation.

Modèle B — BERT avec fine-tuning

Principe du fine-tuning

Dans cette seconde approche, **tous les poids de BERT sont mis à jour** pendant l'entraînement, y compris les couches d'attention. On ajoute en sortie une **tête de classification** (couche `Linear`) qui projette le vecteur `[CLS]` vers les `num_labels` classes.

Le pipeline complet est le suivant :

Texte brut $\xrightarrow{\text{BertTokenizer}}$ Tokens $\xrightarrow{\text{BertForSequenceClassification}}$ Logits $\xrightarrow{\text{CrossEntropyLoss} + \text{backprop}}$

Poids mis à jour

Critère	Modèle A (sans fine-tuning)	Modèle B (avec fine-tuning)
Poids BERT	Gelés	Mis à jour
Temps d'entraînement	Court	Long (GPU recommandé)
Mémoire	Faible	Élevée
Performances attendues	Baseline	Meilleures
Données nécessaires	Peu	Quelques milliers suffisent

Préparation du Dataset PyTorch

PyTorch utilise des objets `Dataset` et `DataLoader` pour gérer l'itération sur les données par batches pendant l'entraînement. Une classe `MentalHealthDataset` encapsule les tenseurs tokenisés et les labels. Le `DataLoader` est configuré avec `batch_size=16` (ou 32 selon la mémoire GPU) et `shuffle=True` pour le train uniquement.

Pourquoi mélanger (`shuffle=True`) ?

Mélanger les exemples à chaque epoch évite que le modèle apprenne l'ordre des données. Cela stabilise la convergence et réduit le risque de sur-apprentissage.

Chargement du modèle et tête de classification

`BertForSequenceClassification` est le modèle officiel Hugging Face pour la classification de texte. Il intègre automatiquement :

- Le `BertModel` de base (12 couches Transformer) ;
- Un `Dropout(0.1)` appliqué au vecteur `[CLS]` ;
- Une couche `Linear(768, num_labels)` comme tête de classification.

Optimiseur : AdamW avec un *learning rate* de 2×10^{-5} , valeur recommandée par l'équipe Google pour le fine-tuning BERT.

Pourquoi AdamW et pas Adam ?

AdamW corrige une erreur dans l'implémentation de la *weight decay* d'Adam classique. Google recommande explicitement AdamW pour le fine-tuning BERT.

Scheduler : un `get_linear_schedule_with_warmup` fait décroître le learning rate linéairement jusqu'à 0 au fil des epochs. Cela évite des mises à jour trop agressives en fin d'entraînement, qui pourraient déstabiliser les poids pré-entraînés.

Boucle d'entraînement

La boucle suit le schéma classique PyTorch, répété sur `num_epochs` epochs (2 à 4 suffisent généralement pour BERT) :

```
Pour chaque epoch :
    model.train()
    Pour chaque batch :
        1. Envoyer le batch sur le device (GPU/CPU)
        2. Passage avant (forward) -> logits et loss
        3. Retropropagation (loss.backward())
        4. Clipper les gradients (clip_grad_norm_, max_norm
            =1.0)
        5. Mise a jour des poids (optimizer.step())
        6. Mise a jour du scheduler (scheduler.step())
        7. Remise a zero des gradients (optimizer.zero_grad())
    Afficher la loss moyenne de l'epoch
```

Gradient clipping

L'appel `clip_grad_norm_(max_norm=1.0)` est essentiel pour BERT : il empêche les gradients d'exploser lors des premières epochs, ce qui peut déstabiliser les poids pré-entraînés et faire diverger l'entraînement.

Temps d'exécution

Environ 15 à 30 minutes par epoch sur GPU (Colab T4), 2 à 3 heures sur CPU. **L'utilisation d'un GPU est fortement recommandée.**

Évaluation

Une fonction `predict(model, dataloader, device)` itère sur les batchs en mode `torch.no_grad()`, applique un `argmax` sur les logits et retourne les tableaux `y_true` et `y_pred`. Les métriques sont ensuite calculées identiquement au Modèle A.

Modèle C — BERT spécialisé + fine-tuning

Pourquoi un modèle spécialisé ?

Le Modèle B part de `bert-base-uncased`, pré-entraîné sur Wikipedia et BooksCorpus — un corpus de texte très éloigné du langage émotionnel ou clinique. Le Modèle C part d'un BERT **déjà fine-tuné sur des tâches d'analyse de sentiments**, ce qui lui donne une meilleure représentation initiale des émotions avant même de voir notre dataset.

`bert-base-uncased` (Wikipedia/BooksCorpus) → Fine-tuning sur sentiments (SST-2, Twitter...) → Modèle spécialisé → Fine-tuning sur notre dataset (santé mentale) → Modèle C final

Critère	BERT-base (Modèle B)	Modèle spécialisé (Modèle C)
Pré-entraînement initial	Wikipedia / BooksCorpus	Wikipedia / BooksCorpus
Fine-tuning intermédiaire	Non	Oui — Sentiments
Point de départ	Général	Spécialisé
Fine-tuning sur notre data	Oui	Oui
Performances attendues	Bonnes	Meilleures (théoriquement)

Modèles disponibles sur Hugging Face

Plusieurs modèles sont candidats, selon les contraintes de ressources :

Modèle	Avantages	Identifiant Hugging Face
DistilBERT SST-2	Léger, 40 % plus rapide	<code>distilbert-base-uncased-finetuned-sst-2-english</code>
RoBERTa Twitter	Entraîné sur texte informel	<code>cardiffnlp/twitter-roberta-base-sentiment</code>
BERT multilingue	Supporte plusieurs langues	<code>nlptown/bert-base-multilingual-uncased-sentiment</code>

DistilBERT — le compromis vitesse/performance

DistilBERT est une version allégée de BERT (66 millions de paramètres contre 110 M) : 40 % plus rapide, 60 % plus léger, avec environ 97 % des performances de BERT-base. Il est particulièrement adapté lorsque les ressources GPU sont limitées.

Adaptation de la tête de classification

Le modèle importé a été fine-tuné pour un certain nombre de classes (souvent 2 ou 3 pour positif/négatif/neutre). Notre dataset possède 7 classes. Il faut donc **remplacer la tête de classification** par une nouvelle couche adaptée.

```
model = AutoModelForSequenceClassification.from_pretrained(  
    model_name ,  
    num_labels=num_labels ,  
    ignore_mismatched_sizes=True      # tete remplacee  
)
```

Le paramètre `ignore_mismatched_sizes=True` est indispensable : la tête originale (ex. 2 classes) est remplacée par une nouvelle tête à `num_labels` classes — les dimensions ne correspondent plus, sans ce paramètre PyTorch lèverait une erreur.

Fine-tuning et évaluation

La boucle d'entraînement est identique à celle du Modèle B. Seul le modèle chargé change. Le tokenizer doit également être celui du Modèle C — chaque modèle Hugging Face possède son propre tokenizer, chargé via `AutoTokenizer` pour garantir la compatibilité.

Comparaison globale des modèles

Tableau récapitulatif

Le tableau ci-dessous synthétisera les performances des cinq modèles (Naive Bayes et K-Means issus du notebook principal, plus les trois variantes BERT). Les valeurs marquées d'un point d'interrogation seront renseignées à l'issue de l'implémentation.

Modèle	Accuracy test	Temps approx.	Remarque
Naive Bayes + TF-IDF	0.6875	Quelques secondes	Référence notebook principal
K-Means + TF-IDF	0.4636	Quelques minutes	Non supervisé
BERT sans fine-tuning (A)	?	Minutes (CPU)	Baseline BERT
BERT avec fine-tuning (B)	?	Heures (GPU)	Fine-tuning complet
BERT spécialisé + FT (C)	?	Heures (GPU)	Meilleur point de départ

Questions d'analyse

À l'issue des expériences, les points suivants seront discutés :

1. **Quel modèle obtient la meilleure accuracy ?** Est-ce le résultat attendu selon la théorie ?
2. **Apport du fine-tuning** (Modèle B vs Modèle A) : à combien estime-t-on le gain en F1 moyen ?
3. **Modèle spécialisé vs BERT-base** (Modèle C vs Modèle B) : le modèle pré-entraîné sur des sentiments apporte-t-il un avantage mesurable ?
4. **Compromis coût/performance** : si le déploiement en production est contraint en temps de réponse, quel modèle recommander et pourquoi ?
5. **Classes difficiles** : les mêmes classes (Personality disorder, Stress, confusion Depression/Suicidal) posent-elles problème aux modèles BERT comme aux méthodes classiques ?

Points de vigilance

- **Limite de 512 tokens** : dans notre dataset, la majorité des textes dépassent rarement 128 tokens après nettoyage, ce qui limite l'impact de la troncature. La valeur `max_length=128` est donc un bon compromis.
- **Déséquilibre des classes** : comme dans le notebook principal, Depression est sur-représentée. Les métriques macro (F1 macro) doivent être privilégiées pour ne pas masquer les mauvaises performances sur les classes minoritaires.
- **Confusion Depression/Suicidal** : ce problème persistant dans toutes les méthodes classiques pointait vers une limite des données. BERT, grâce à ses représentations contextuelles, pourrait partiellement atténuer cette confusion — à vérifier expérimentalement.
- **Reproductibilité** : les seeds sont fixées à 42 pour `random`, `numpy` et `torch` afin de garantir la reproductibilité des résultats entre exécutions.

Références

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv :1810.04805.
- Wolf, T. et al. (2020). *Transformers : State-of-the-Art Natural Language Processing*. Hugging Face. <https://huggingface.co/transformers>
- Sentiment Analysis for Mental Health — Kaggle. <https://www.kaggle.com/datasets/suchintikasarkar/sentiment-analysis-for-mental-health>